SE1012 - PROGRAMMING METHODOLOGY

# ADVANCED NAVAL BATTLE SIMULATOR

REPORT

IT23750760
D. N. GUNAWARDHANA

**INTRODUCTION • OBJECTIVE • METHODOLOGY • IMPLEMENTATION • CHALLENGES & SOLUTIONS**
**RESULTS • REFLECTION • CONCLUSION**

# INTRODUCTION

30 days ago, we were tasked to set sail on a challenging university assignment: to develop a naval battle simulator, programmed in C. Our mission was to simulate battles between a lone Battleship and a swarm of Escort Ships. This program allows the user to act as the captain, configuring the battlefield dimensions, selecting the battleship type and the number of escort ships etc. This simulator provides a platform to explore and analyze naval combat strategies within a controlled digital environment.

# OBJECTIVE

The objective of this naval battle simulator is to make the user in charge of the battleship and emulate a combat between the battleship and the escort ships. This entails a two-fold approach: maximizing the elimination of escort ships while minimizing the damage received by the escort ships. This simulator enables the user to explore and refine both offensive and defensive naval battle strategies.

# METHODOLOGY

This report details the development methodology used to create the naval battle simulator program. The development process adopted a sequential approach, progressively adding features in two main parts, each further divided into subparts A, B and C.

## Part 1: Establishing the Core Functionality

### Part 1 - A: Setting the Stage

The first stage of developing the simulator focuses on establishing the fundamentals of the naval battlefield. Assumptions are made to simplify the simulation including, instantaneous shootings, simultaneous attacks from both parties and additionally, a single hit is assumed to be sufficient to destroy any ship. This version populates the canvas with the required number of escort ships, simulates the battle, calculates the number of destroyed escort ships, checks whether the battleship is attacked and prints the final battlefield. It also saves the initial conditions, simulation statistics and the final conditions in three separate text files.

### Part 1 - B: Introducing Movement & Gun Malfunction

This subpart incorporates a movement for the battleship along a predefined path. It also introduces a scenario where the battleship's gun can malfunction, limiting its firing angle. This version simulates the battle for each point along the path or until the battleship is attacked. The results are saved in text files just as before.

### Part 1 - C: Enhancing Realism

This version introduces a more realistic element: The "Impact Power" of escort ships, representing the damage made on the battleship with a single hit. The simulations of Part 1 -A & B are rerun with this new feature, keeping track of the cumulative damage sustained by the battleship.

## Part 2: Advanced Features and Strategies (Optional)

### Part 2 - A: Reload Time & Attacking Order

This section tackles a more complex challenge. It factors in the time required for the battleship to reload its gun between shots. Here, I developed a strategy for the battleship to determine the optimal attacking order, aiming to maximize enemy casualties while minimizing damage on itself. The program reruns the simulations from Part 1 - A, B & C with the reload time feature and the attacking strategy implemented. The results are saved in text files as usual.

### Part 2 - B: Continuous Escort Ship Fire

In this version, the escort ships are now able to fire more than once. Similar to Part 2 - A, now the simulation introduces a reload time for escort ships as well. The reload time is a unique time for each escort ship type. The program then reruns all the simulations from Part 1 implementing the new features as well as the attack strategy from the previous version.

### Part 2 - C: Weapon Degradation

This final version introduces an even more realistic element. The more a ship fires its gun, the weaker its attacks become. The program conducts the simulations from Part 1 - C with this feature, monitoring the impact power of each ship throughout the battle.

This sequential development approach allows for a clear understanding of the simulator’s functionality and facilitates the gradual introduction of advanced features and strategic elements.

# IMPLEMENTATION

The implementation of this simulator involved careful handling of the programming language, logic design and the implementation of novel features to enhance the realism and the effectiveness of the simulation.

The simulation was implemented using the C programming language. C provided the necessary control over memory allocation and efficient data manipulation required for simulating complex battlefield scenarios.

## Logic Design:

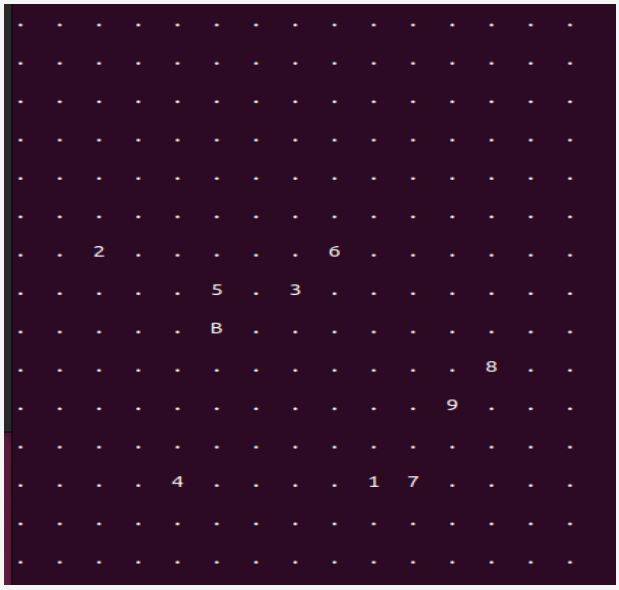
1. **Ship Representation:**
   The battleship and the escort ships were represented as structures, each containing common attributes such as position, firing velocity, firing angle range and as well as unique attributes such as impact power, ID etc.

```c
//The structure of the Battleship
typedef struct {
  char shipname[20]; //1
  char notation[2];  //2
  char gunname[30]; //3
  int maxv;  //4
  int x;  //5
  int y;  //6
} BattleShip;
```

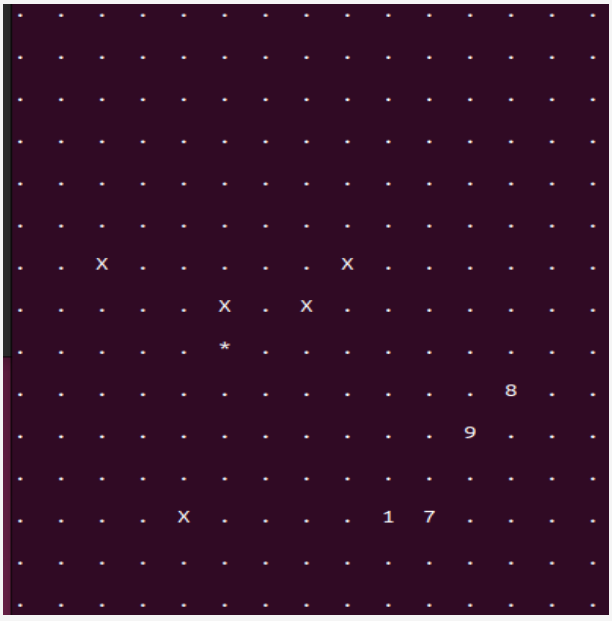
```c
//The structure of an Escort Ship
typedef struct {
  char notation[4]; //1
  char shipname[30]; //2
  char gunname[30];  //3
  float impactpower; //4
  int anglerange; //5
  int minangle; //6
  int minv; //7
  float maxv; //8
  int x; //9
  int y; //10
  int id; //11
  int status; //12
  int maxangle; //13
} EscortShip;
```

## 2. Battlefield Environment:

The battlefield was modeled as a grid, with each grid cell representing a potential position for ships to occupy. This enabled spatial calculations and interactions between ships.



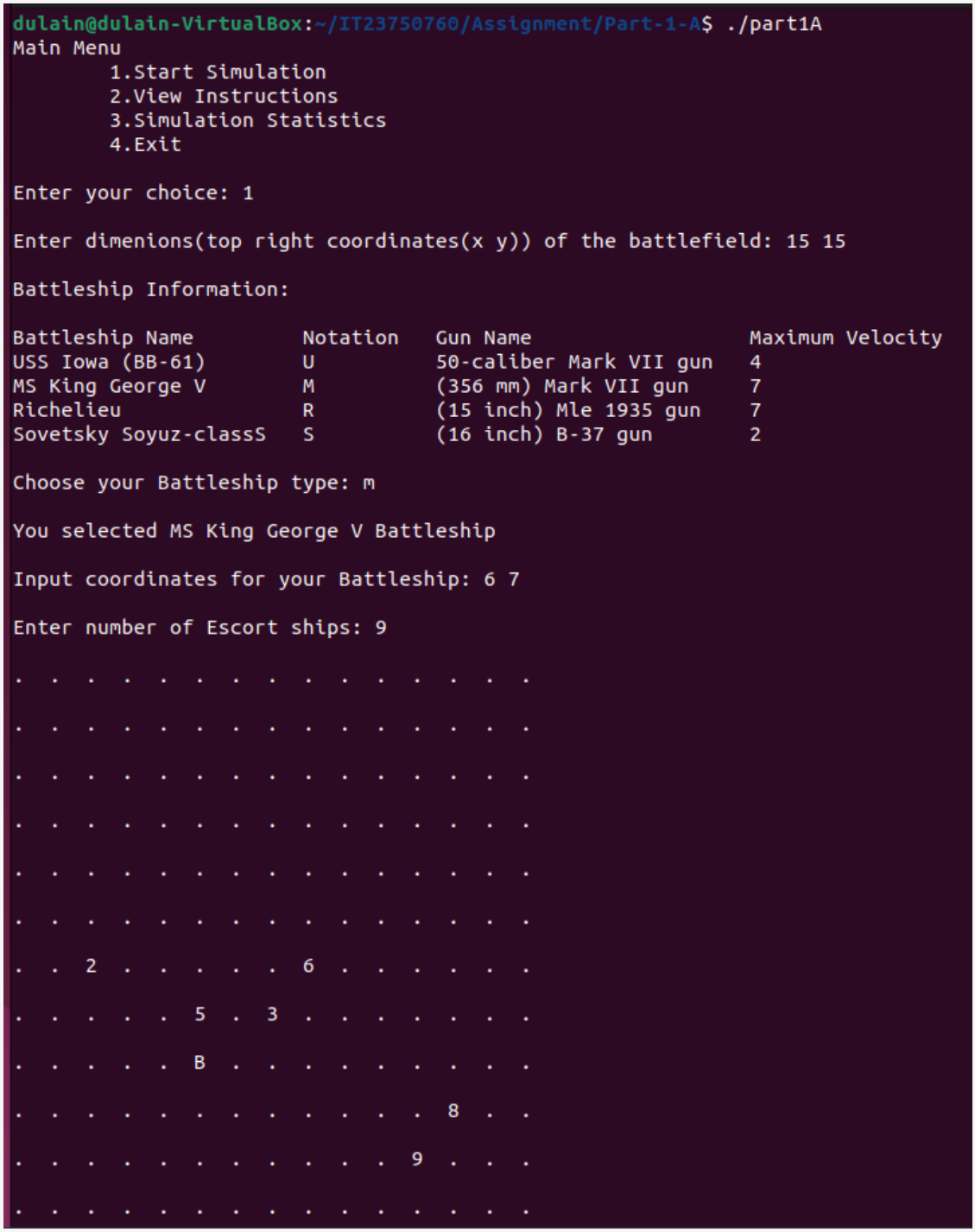

The Battleship was denoted by a “**B**”
The Escort Ships were denoted by their unique **ID** number.




The destroyed Battleship was denoted by an “ * “
Destroyed Escort Ships were denoted by an “**X**“

## 3. Simulation Flow:

The simulation followed a sequential flow, where ships were initialized, placed on the battlefield, and then engaged in a combat, based on predefined rules and conditions. Iterative loops controlled the progression of time steps, allowing for dynamic movement and interactions between ships.



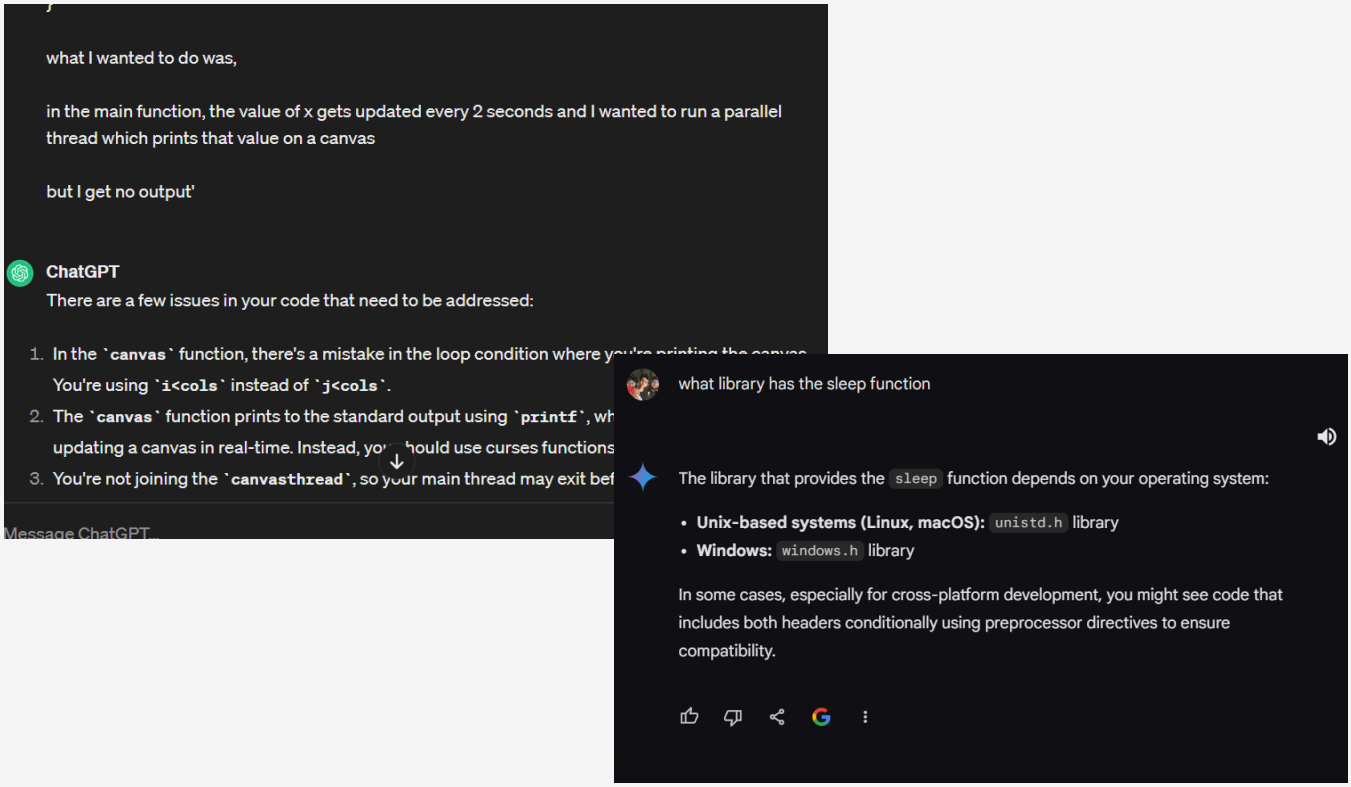
# CHALLENGES & SOLUTIONS

During the development process of this simulator, a *multi-dimensional* array of challenges were encountered ranging from mastering new concepts in C, to using clever logic for simulation mechanics.

## Learning New Concepts

Undoubtedly, the biggest challenge I encountered during the development was to familiarize myself with entirely new concepts in C programming. Starting from random number generators, seeding the random number generator, handling multiple text files and opening them through user commands were a few to mention. Understanding pointers, memory allocation, arrays of structures containing more than 10 attributes, functions inside functions and function pointers were the initial hurdles.

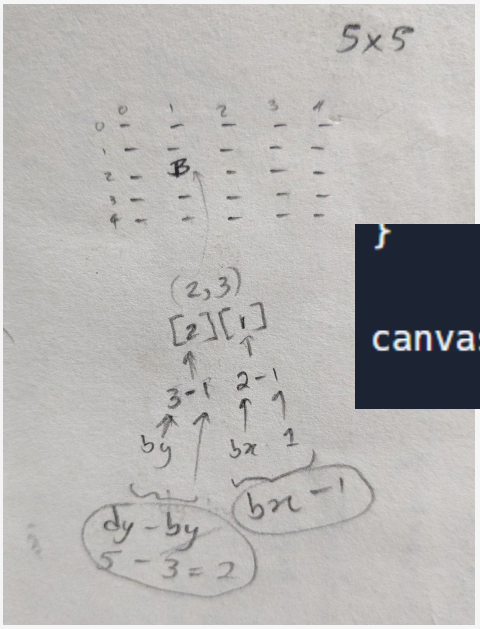
To overcome this challenge, **extensive** self-study using resources such as AI (ChatGPT & Gemini), YouTube tutorials and online forums were utilized. These tools coupled with self experimenting proved invaluable in clarifying complex concepts and addressing specific questions.



## Clever Logics

Having to figure out seemingly minor yet crucial algorithms required for implementing advanced features emerged as my favorite challenge among them all, particularly because the satisfaction and the sense of accomplishment I get upon devising such logic successfully.

Breaking down the problem into smaller components followed by sketching, testing and debugging a simplified version of the problem proved to be the most effective approach for me in resolving logical problems.




```
}

canvas[dy - Bship.y][Bship.x - 1] = -1; //B
```

```
//This function generates the path of the Battleship (coordinates)
void BshipPath(PathXY path[], EscortShip Eships[], int eships, BattleShip Bship, int dx, int dy, int points){

  // Initializing the first point of the path as the Battleship's initial position
  path[0].x = Bship.x;
  path[0].y = Bship.y;

  for (int i = 1; i < points; i++) {
    //declaring 2 variables to temporarily store the next coordinates of the path
    int newX, newY;

    do {
      // Generating the next point of the path
      newX = path[i - 1].x + randomnumber(-10, 10);
      newY = path[i - 1].y + randomnumber(-10, 10);
    } while (newX <= 0 || newY <= 0 || newX >= dx || newY >= dy); //checking if the new point is within the boundaries of the battlefield

     //checking if the point collides with an Escort Ship
     for (int j = 0; j < eships; j++) {
       //if a collision occurs, generate a new point
       if (newX == Eships[j].x && newY == Eships[j].y) {
         newX = path[i - 1].x + randomnumber(-10, 10);
         newY = path[i - 1].y + randomnumber(-10, 10);
         j = -1; //restart collision check loop
       }
     }
    //assigning the new point to the path only after all the eligibility tests are done
    path[i].x = newX;
    path[i].y = newY;
  }
}
```

## Error Identification and Resolution

Among the challenges encountered, error handling proved to be the most demanding. Solving close to a hundred segmentation faults was particularly frustrating yet informative. Moreover, solving errors spanning from logical missteps to syntax errors took a heavy mental strain on me.

Utilizing AI tools and debugging helped in identifying errors. One particular method proved to be the most effective to me, which was to make a suspicious execution a comment and then run the rest of the simulation. This approach helped me greatly in identifying mistakes even more quickly.

```
//declaring and allocating memory for an int array to print the battlefield
int **canvas = (int **)malloc(dy * sizeof(int *));
for(int i=0; i<dy; i++){
  canvas[i] = (int *)malloc(dx * sizeof(int));
}
```

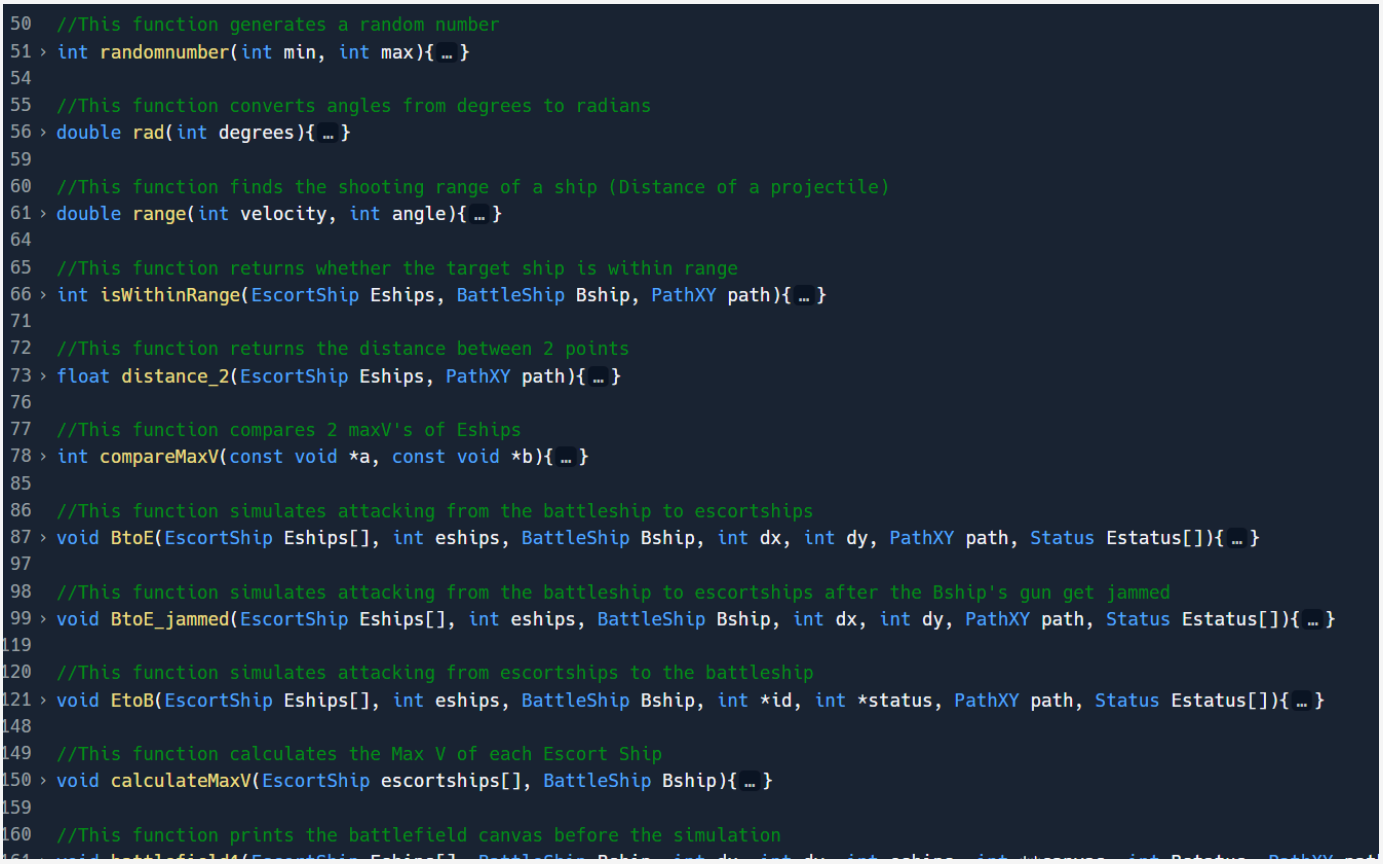
```
//Free the allocated memory for the 2D array (important!)
for (int i = 0; i < dy; i++) {
    free(canvas[i]);
}
free(canvas);
```

```
//a separate variable to store the status of the Battleship(using the Bship structure for this didn't work)
int Bstatus = 0;

//a separate array to store the status of Escortships(using the Eships structure for this didn't work)
Status Estatus[eships];
```

## Being Organized

Maintaining a well organized and well commented code presented another challenge. With over 10 functions intertwined and utilized with each other, maintaining clarity within the program was crucial hence I had to be organized from the very beginning of this journey.



```
50  //This function generates a random number
51  int randomnumber(int min, int max){ … }
54
55  //This function converts angles from degrees to radians
56  double rad(int degrees){ … }
59
60  //This function finds the shooting range of a ship (Distance of a projectile)
61  double range(int velocity, int angle){ … }
64
65  //This function returns whether the target ship is within range
66  int isWithinRange(EscortShip Eships, BattleShip Bship, PathXY path){ … }
71
72  //This function returns the distance between 2 points
73  float distance_2(EscortShip Eships, PathXY path){ … }
76
77  //This function compares 2 maxV's of Eships
78  int compareMaxV(const void *a, const void *b){ … }
85
86  //This function simulates attacking from the battleship to escortships
87  void BtoE(EscortShip Eships[], int eships, BattleShip Bship, int dx, int dy, PathXY path, Status Estatus[]){ … }
97
98  //This function simulates attacking from the battleship to escortships after the Bship's gun get jammed
99  void BtoE_jammed(EscortShip Eships[], int eships, BattleShip Bship, int dx, int dy, PathXY path, Status Estatus[]){ … }
119
120 //This function simulates attacking from escortships to the battleship
121 void EtoB(EscortShip Eships[], int eships, BattleShip Bship, int *id, int *status, PathXY path, Status Estatus[]){ … }
148
149 //This function calculates the Max V of each Escort Ship
150 void calculateMaxV(EscortShip escortships[], BattleShip Bship){ … }
159
160 //This function prints the battlefield canvas before the simulation
```

# RESULTS

The outcome of this simulation is a comprehensive view of how naval warfare would take place in real life. Throughout the simulation process, relevant data points and statistics were carefully recorded to provide a quantitative assessment over their performance. These results serve as a foundation for evaluating the effectiveness of various strategies and tactics deployed during this simulation, guiding future refinements.

In addition to the program itself, this assignment taught us valuable lessons in self-learning. We delved into new concepts beyond what was covered in class, honing our ability to grasp new and complex ideas independently. This experience emphasized the importance of self organization in our work. By taking initiative and exploring beyond the lecture notes, we enhanced our understanding and the readiness to tackle challenges head-on.

# REFLECTION

Reflecting the overall experience of working on this assignment, I'd summarize it as a "Life changing journey". Engaging in this program demanded intense mental focus and dedication, consuming my attention for **weeks** on end. During this time, I immersed myself in learning concepts that I previously did not even know existed, which changed my approach to problem-solving. This experience not only expanded my knowledge but also reshaped my logical thinking process. Life will unironically be different after completing this assignment.

Despite not completing all the optional parts, I made an effort to progress up to Part 2-A. This involved learning entirely new concepts such as **multi-threading** and **ncurses** in C programming. Even though I could not use these concepts successfully in this simulation, I gained a clear understanding of the purpose and practical applications of these concepts. While some might argue that this assignment was way too challenging for a first year student, I believe it serves as a distinguishing factor among students pursuing similar degree programs and the assignment's encouragement of self-learning and exploration of new concepts is highly commendable. Overall, I consider this assignment to be one of the most beneficial and important experiences during my academic journey.

# CONCLUSION

My version of this simulator still has room for refinements but due to time constraints, I held onto the 90% rule and left it as it is. If I had more time to fine tune this project, I'd include features such as, not printing the battlefield if the user inputs of the dimensions are too large, regenerate an escort ship position if the generated position is already taken by the battleship etc.

Additionally, this simulator has potential for additional features that could further elevate its effectiveness and realism such as, incorporating dynamic weather conditions and environmental factors, integrating more sophisticated path finding algorithms and graphics.

These changes have the potential to significantly impact the simulator's effectiveness by providing the user with a more immersive and a challenging experience.

Throughout this report, I've outlined the methodology, implementation details, challenges faced and the results of the simulator. I've mentioned the importance of self-learning and problem solving skills as well as the significance of exploring new concepts in programming.

In the context of programming methodology, this assignment highlights the importance of practical application and hands-on experience in developing theoretical knowledge. By engaging in projects like this, students can develop a deeper understanding of programming principles and enhance their ability to tackle real-world challenges effectively.